

RDD Analysis in Spark

I'll give you a complete assignment-style solution using PySpark RDDs, with code for each section and simple explanations you can write in your notebook or run in Jupyter/Colab.

Below is a complete **Apache Spark RDD assignment solution**.

Apache Spark RDD Dataset Analysis

Before Starting

Install PySpark if needed:

```
Python
!pip install pyspark
```

Import and create SparkContext:

```
Python
from pyspark import SparkContext
import re
```

```
Python
sc = SparkContext.getOrCreate()
```

SparkContext is the main entry point for working with RDDs.

Section A – RDD Creation and Partitioning

1. Create an RDD from integers 1 to 10 using multiple partitions

```
Python
numbers = list(range(1, 11))

rdd_numbers = sc.parallelize(numbers, 3)

print(rdd_numbers.collect())
print("Number of partitions:", rdd_numbers.getNumPartitions())
```

Explanation

```
Python
sc.parallelize(numbers, 3)
```

creates an RDD from a Python list.

Here, 3 means Spark will divide the data into **3 partitions**.

Output example:

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Number of partitions: 3
```

2. Load dataset and display number of partitions

Assume the book file is saved as:

`pride.txt`

Load file:

```
Python
book_rdd = sc.textFile("pride.txt")

print("Number of partitions:", book_rdd.getNumPartitions())
print(book_rdd.take(5))
```

Explanation

```
Python
sc.textFile("pride.txt")
```

loads the text file as an RDD.

Each line becomes one element of the RDD.

3. Load dataset with custom number of partitions

```
Python
book_rdd_4 = sc.textFile("pride.txt", 4)

print("Number of partitions:", book_rdd_4.getNumPartitions())
```

Explanation

Here Spark tries to create **4 partitions** while reading the file.

Partitioning helps Spark process data in parallel.

Section B – Text Analysis

Use the book text dataset: **Pride and Prejudice**

4. Load dataset and perform word-level analysis

```
Python
book_rdd = sc.textFile("pride.txt")

Split lines into words:

Python
words_rdd = book_rdd.flatMap(lambda line: line.split())

print(words_rdd.take(20))
```

Explanation

`flatMap()` is used because one line contains many words.

Example:

"This is Spark"

becomes:

```
["This", "is", "Spark"]
```

5. Remove irrelevant or uninformative tokens

We remove very short words like `a`, `I`, `to`, etc.

Python

```
filtered_words = words_rdd.filter(lambda word: len(word) > 2)

print(filtered_words.take(20))
```

Explanation

Python

`filter()`

keeps only words satisfying a condition.

Here:

Python

```
len(word) > 2
```

keeps words having more than 2 characters.

6. Normalize the text

Normalize means:

- convert to lowercase
- remove punctuation
- keep only alphabets

Python

```
clean_words = book_rdd.flatMap(lambda line: line.split()) \
    .map(lambda word: word.lower()) \
    .map(lambda word: re.sub('[^a-z]', '', word)) \
    .filter(lambda word: len(word) > 2)

print(clean_words.take(30))
```

Explanation

Example:

"Elizabeth,"

becomes:

"elizabeth"

This makes counting accurate.

Without normalization:

Elizabeth
elizabeth
Elizabeth,

would be counted as different words.

7. Generate frequency count of all valid words

Python

```
word_pairs = clean_words.map(lambda word: (word, 1))
word_counts = word_pairs.reduceByKey(lambda a, b: a + b)
print(word_counts.take(20))
```

Explanation

First we create key-value pairs:

Python

```
('elizabeth', 1)
```

Then `reduceByKey()` adds values for the same word.

Example:

```
("elizabeth", 1)
("elizabeth", 1)
("elizabeth", 1)
```

becomes:

```
("elizabeth", 3)
```

Display top 20 most frequent words

Python

```
top_words = word_counts.takeOrdered(20, key=lambda x: -x[1])
for word, count in top_words:
    print(word, count)
```

Explanation

Python

```
takeOrdered(20, key=lambda x: -x[1])
```

sorts words by frequency in descending order.

8. Display sample outputs from processed data

Python

```
print("Sample clean words:")
print(clean_words.take(20))

print("Sample word count pairs:")
print(word_counts.take(20))
```

9. Explore key-value dataset and retrieve meaningful subsets

Words starting with "love"

Python

```
love_words = word_counts.filter(lambda x: x[0].startswith("love"))
print(love_words.collect())
```

Words appearing more than 100 times

Python

```
frequent_words = word_counts.filter(lambda x: x[1] > 100)
print(frequent_words.take(20))
```

Sort words alphabetically

Python

```
sorted_words = word_counts.sortByKey()
print(sorted_words.take(20))
```

Section C – Log File Analysis

Use NASA log file dataset.

Assume file name:

nasa.log

Load log file:

Python

```
log_rdd = sc.textFile("nasa.log")

print(log_rdd.take(5))
print("Number of partitions:", log_rdd.getNumPartitions())
```

10. Extract client identifiers / IP addresses

In web server logs, the client identifier is usually the first value in each line.

Example log line:

```
199.72.81.55 - - [01/Jul/1995:00:00:01 -0400] "GET /history/apollo/ HTTP/1.0" 200 6245
```

Client/IP:

```
199.72.81.55
```

Code:

Python

```
clients = log_rdd.map(lambda line: line.split()[0])
print(clients.take(10))
```

11. Count requests made by each unique client

Python

```
client_pairs = clients.map(lambda client: (client, 1))
client_counts = client_pairs.reduceByKey(lambda a, b: a + b)
print(client_counts.take(10))
```

12. Display small sample of result

Python

```
top_clients = client_counts.takeOrdered(10, key=lambda x: -x[1])
for client, count in top_clients:
    print(client, count)
```

Explanation

This shows clients who made the highest number of requests.

Section D – Set Operations

13. Create two RDDs with overlapping values

Python

```
rdd1 = sc.parallelize([1, 2, 3, 4, 5])
rdd2 = sc.parallelize([4, 5, 6, 7, 8])
```

14. Perform union and intersection

Union

Python

```
union_rdd = rdd1.union(rdd2)
print("Union result:", union_rdd.collect())
```

Output:

```
[1, 2, 3, 4, 5, 4, 5, 6, 7, 8]
```

Union combines both RDDs.

It does not remove duplicates automatically.

Intersection

```
Python
intersection_rdd = rdd1.intersection(rdd2)

print("Intersection result:", intersection_rdd.collect())
```

Output:

```
[4, 5]
```

Intersection returns common values.

Union without duplicates

```
Python
unique_union = rdd1.union(rdd2).distinct()

print("Unique union:", unique_union.collect())
```

Output:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Section E – Data Storage and Retrieval

16. Save processed outputs

Save word frequencies

```
Python
word_counts.saveAsTextFile('output_word_counts')
```

Save client request counts

```
Python
client_counts.saveAsTextFile('output_client_counts')
```

Important

Before running again, delete old output folders. Spark gives error if output folder already exists.

In Linux/Colab:

```
Python
!rm -r output_word_counts
!rm -r output_client_counts
```

Then rerun save commands.

17. Retrieve and display saved entries

Read saved word counts

Python

```
saved_word_counts = sc.textFile("output_word_counts")
print(saved_word_counts.take(10))
```

Read saved client counts

Python

```
saved_client_counts = sc.textFile("output_client_counts")
print(saved_client_counts.take(10))
```

Complete Combined Code

Python

```
from pyspark import SparkContext
import re

sc = SparkContext.getOrCreate()

# Section A
numbers = list(range(1, 11))
rdd_numbers = sc.parallelize(numbers, 3)

print("Integer RDD:", rdd_numbers.collect())
print("Integer RDD partitions:", rdd_numbers.getNumPartitions())

book_rdd = sc.textFile("pride.txt")
print("Book partitions:", book_rdd.getNumPartitions())
print("Book sample:", book_rdd.take(5))

book_rdd_4 = sc.textFile("pride.txt", 4)
print("Book custom partitions:", book_rdd_4.getNumPartitions())

# Section B
clean_words = book_rdd.flatMap(lambda line: line.split()) \
    .map(lambda word: word.lower()) \
    .map(lambda word: re.sub('[^a-z]', "", word)) \
    .filter(lambda word: len(word) > 2)

print("Clean words sample:")
print(clean_words.take(20))

word_counts = clean_words.map(lambda word: (word, 1)) \
    .reduceByKey(lambda a, b: a + b)

print("Word count sample:")
print(word_counts.take(20))

print("Top 20 words:")
top_words = word_counts.takeOrdered(20, key=lambda x: -x[1])

for word, count in top_words:
    print(word, count)

print("Words starting with love:")
print(word_counts.filter(lambda x: x[0].startswith("love")).collect())

print("Words appearing more than 100 times:")
print(word_counts.filter(lambda x: x[1] > 100).take(20))

# Section C
log_rdd = sc.textFile("nasa.log")

print("Log partitions:", log_rdd.getNumPartitions())
print("Log sample:")
print(log_rdd.take(5))
```



```
clients = log_rdd.map(lambda line: line.split()[0])

client_counts = clients.map(lambda client: (client, 1)) \
    .reduceByKey(lambda a, b: a + b)

print("Client count sample:")
print(client_counts.take(10))

print("Top 10 clients:")
top_clients = client_counts.takeOrdered(10, key=lambda x: -x[1])

for client, count in top_clients:
    print(client, count)

# Section D
rdd1 = sc.parallelize([1, 2, 3, 4, 5])
rdd2 = sc.parallelize([4, 5, 6, 7, 8])

union_rdd = rdd1.union(rdd2)
intersection_rdd = rdd1.intersection(rdd2)
unique_union = rdd1.union(rdd2).distinct()

print("Union:", union_rdd.collect())
print("Intersection:", intersection_rdd.collect())
print("Unique union:", unique_union.collect())

# Section E
word_counts.saveAsTextFile('output_word_counts')
client_counts.saveAsTextFile('output_client_counts')

saved_word_counts = sc.textFile("output_word_counts")
saved_client_counts = sc.textFile("output_client_counts")

print("Saved word counts:")
print(saved_word_counts.take(10))

print("Saved client counts:")
print(saved_client_counts.take(10))
```

Key RDD Operations Used

Operation	Meaning
<code>parallelize()</code>	Creates RDD from Python list
<code>textFile()</code>	Loads text file into RDD
<code>getNumPartitions()</code>	Shows number of partitions
<code>flatMap()</code>	Splits one record into many records
<code>map()</code>	Transforms each record
<code>filter()</code>	Removes unwanted data
<code>reduceByKey()</code>	Aggregates values by key
<code>take()</code>	Displays sample records
<code>collect()</code>	Brings full RDD data to driver
<code>union()</code>	Combines two RDDs
<code>intersection()</code>	Finds common elements
<code>distinct()</code>	Removes duplicates
<code>saveAsTextFile()</code>	Saves RDD output to folder

Final Conclusion

In this assignment, we used Apache Spark RDDs to analyze real-world text and log datasets. We created RDDs, checked partitions, cleaned and normalized text data, counted word frequencies, extracted client IP addresses from log files, counted requests, performed set operations, and saved the results. This demonstrates how RDDs support distributed data processing using transformation and action operations.